

How header files let multiple .c files share one function (no duplicate code)

Say you write `ft_strlen` once in `a.c`, and you want `b.c`, `z.c`, and `y.c` to all be able to call it. Here's exactly how that works, from source files to a running program.

1. The header holds the declaration only, not the code

```
// mylib.h
#ifndef MYLIB_H
# define MYLIB_H

int ft_strlen(char *s);

#endif
```

2. The real implementation lives in exactly one .c file

```
// a.c
#include "mylib.h"

int ft_strlen(char *s)
{
    int i = 0;
    while (s[i])
        i++;
    return (i);
}
```

3. Any other file that wants to use it just includes the header

```
// b.c
#include "mylib.h"
#include <stdio.h>

void print_len_b(char *str)
{
    printf("b.c says length is %d\n", ft_strlen(str));
}
```

```
// z.c
#include "mylib.h"
#include <stdio.h>

void print_len_z(char *str)
{
    printf("z.c says length is %d\n", ft_strlen(str));
}
```

```
// main.c
#include "mylib.h"

void print_len_b(char *str);
void print_len_z(char *str);

int main(void)
{
    print_len_b("hello");
    print_len_z("world!");
    return (0);
}
```

4. Each .c file compiles SEPARATELY into its own object file

```
cc -c a.c -o a.o cc -c b.c -o b.o cc -c z.c -o z.o cc -c main.c -o main.o
```

When `b.c` is compiled, the header gives the compiler just enough info (the function's signature) to check the call `ft_strlen(str)` is valid. The compiler does NOT need to see the real implementation — it just trusts "this exists somewhere" and leaves an unresolved reference to the symbol `ft_strlen` inside `b.o`. Same thing happens for `z.o` and `main.o`. Only `a.o` actually contains the real compiled machine code for `ft_strlen`.

5. Linking connects everything into one executable

```
cc a.o b.o z.o main.o -o program
```

The linker scans every `.o` file, finds all the unresolved `ft_strlen` references (from `b.o`, `z.o`), and wires them directly to the one real definition sitting in `a.o`. Now every call to `ft_strlen`, no matter which file it came from, points to the same single compiled function.

6. Run it

```
./program b.c says length is 5 z.c says length is 6
```

TL;DR: the header is a compile-time contract — it lets each `.c` file compile independently without seeing each other's source. The actual connection between "b.c calling `ft_strlen`" and "a.c's real code" only happens later, at the link step, when the linker matches every reference to its one real definition. This is how you avoid retyping the function (or its prototype) in every file that needs it — write it once, declare it once, use it everywhere.